Orderflow Engine — Solution walkthrough

Jesús Ariel González Bonilla

2026-10-17

Question this document answers: how was the assignment solved, step by step, and which engineering decisions were taken along the way?

This walkthrough is written for the evaluator. It follows the work in the order it happened: reading the assignment, writing the specification, planning, building the pricing engine and the order confirmation, proving them, and delivering. Every number in it comes from a repository file or from the command named next to it. The verified state lives in one place, orderflow-engine-docs/handoff/HANDOFF.md §3, and this document quotes it.

1. The assignment in one page

The assignment (orderflow-engine-docs/assignment/) describes a B2B marketplace where shopkeepers buy from beverage distributors. One multi-tenant platform serves several countries (Colombia, Ecuador, Guatemala, Peru) and several brands. An order goes through catalog, cart, checkout (the quote), confirmation and history. Five commercial levers act on the price and can coexist in one order: tier discounts, combos, gifts, taxes and credit. The window is five days, and the language is Go, Java (Spring Boot or WebFlux) or NestJS.

Part	What it asks	Where this document answers it
A1. Pricing engine	An endpoint or use case that takes a cart and returns the breakdown per line and totals: gross subtotal, discounts with their origin, gifts with their quantity, taxable base, taxes and total. The levers coexist, and their order must be defined and justified. A credit order must say whether it fits the available credit	§5
A1. Non-functional core	The quoted total and the confirmed total must be identical, even if prices or levers change in between. No floating point for money, with an explicit type, decimal scale, rounding rule and rounding points. Currency, scale and tax rates cannot be hard-coded	§5, §6.1
A1. Golden tests	Six required cases: a discount in two brackets; a combo whose products do not appear as a loose discount; a gift with its quantity; a combo with taxes; a credit order over the limit; quote and confirm give the same total	§5.5
A2. Order confirmation	Orders were confirmed, an order number was returned, and later the order was missing from the history, in three different releases. POST /orders takes an idempotency key, so retries	§6

	never duplicate an order or consume credit twice. A 201 means the order exists and can be read. Push and ERP effects are never lost or duplicated, also when the external system is down. The consistency model and what it sacrifices are documented	
A3. ADR	At most one page: decisions and discarded alternatives, what was not implemented on purpose, what to check in a code review, and one risk with its mitigation	§7
Delivery	A Git repository with the code and tests, a single command to run them, a README with how to run, the lever order, the rounding policy and the consistency model, the ADR, and a readable commit history	§11, §14
Optional front	One order-summary screen in Angular or Flutter, at most one hour of work. Three defects must not happen: truncated text, controls hidden by the system bar or the keyboard, and images of one grid scaled differently. A bonus for explaining how to prevent them at component level	§8

The assignment itself calls the non-functional requirements of A1 the heart of the exercise. The design started there: a sealed quote for "quoted equals confirmed", a decimal money type with per-line rounding, and country data instead of constants.

2. Reading the problem: ambiguities and assumptions

Before any code, the assignment was read sentence by sentence. Each sentence that allowed two readings became an assumption with a stable ID in orderflow-engine-docs/spec/SPEC.md §13, and each assumption is enforced by a business rule (BR-xx) and a test.

ID	Ambiguity in the assignment	Assumption taken	Why
A-01	Tier discounts are "acumulativas por tramos" (cumulative by brackets)	Marginal brackets: each percentage applies only to the units inside its bracket, and percentages never stack	The brackets accumulate segment by segment, as tax brackets do. Golden test 1 pins this reading with exact numbers
A-02	A SKU can be covered by a SKU rule and by a family rule at the same time	A SKU rule removes that SKU from family rules	The specific rule wins, and no unit gets two tier discounts (TierPrecedenceTest)
A-03	The order in which	Normalized SKU order	The same cart always prices

	units of a family fill the brackets		the same, whatever the order of its lines (BR-00)
A-04	Whether combo units and gift units count toward tiers	They are excluded from thresholds and tier discounts	No double dipping: a unit priced by a combo is not discounted again (golden test 2)
A-05	How long a quote is valid	15 minutes by default, set per country as data	It bounds how long a sealed price is honored, the risk of the ADR
A-06	Who the caller is: the assignment says nothing about authentication	Authentication is out of scope. Tenant and customer come from the X-Tenant-Id and X-Customer-Id headers	A gateway would inject them. The web client's store picker stands in for a login
A-07	How a combo appears in the breakdown	One grouped discount entry per combo. Line allocations exist only to derive each line's taxable base	Golden test 2 forbids showing the combo's products as loose discounts, and tax still needs a base per line
A-08	Whether credit is reserved when the quote is made	No. At quote time the check is advisory	The binding check runs once, inside the confirmation transaction. Two open quotes that both fit are settled there: one confirms, the other gets 409
A-09	Cancellation and credit release	Out of scope	The assignment does not ask for them
A-10	What "a purchase condition" for a gift means	A minimum paid quantity of a SKU or family, or a minimum gross subtotal before discounts. A gift line is always separate from a paid line	A gift is triggered by what is bought and never changes what is paid for product

Some questions are asked of the candidate directly. Each answer is a rule of the specification:

Question in the assignment	**Answer**	**Where**
Which money type, scale and rounding rule?	Money holding a BigDecimal and a currency policy; scale and rounding mode from country data; HALF_UP in the four seeded countries	SPEC §4
Round per line, per total or per tax?	Per line only. Totals are sums of rounded lines and are never rounded again	SPEC §4
What does the gift's tax	The country's gift tax policy decides: EXEMPT	SPEC

implication mean?	or TAXED on the gift's reference value	BR-04
What does a 201 guarantee?	The order row is committed and readable before the 201 is serialized (invariant I-201)	SPEC §7.2
Which consistency model, and what is sacrificed?	Strong for order, credit, key and outbox in one database; eventual for push and ERP	SPEC §7.5

3. Specification first

The specification was written and merged before the first line of service code. The first pull requests of the documentation repository on 2026-10-14 are the constitution and the specification, then the plan, tasks and contract, then the data model and architecture. The service code started on 2026-10-15.

3.1 The constitution

orderflow-engine-docs/spec/constitution.md holds 15 non-negotiable principles. Each one names how it is verified, because "a principle without a verification is a wish". The ones that shaped the solution most:

Article	Principle	Verified by
P-01	Money is never a binary floating-point number	DomainPurityArchTest, MoneyTest
P-02	The quoted total is the confirmed total: the order references the sealed quote and stores no amount	QuoteConfirmRegressionIT (golden test 6)
P-03	A 201 is sent only after the commit of the transaction that created the order	ConfirmOrderCommitOrderingIT, OrderReadYourWritesIT
P-04	Every external effect goes through the outbox, written in the order transaction	OutboxWriteIT, OutboxRelayIT
P-05	No remote call inside a database transaction	TransactionBoundaryArchTest
P-06	The pricing domain is pure: no Spring, JDBC, HTTP or clock	DomainPurityArchTest, the golden tests
P-07	No country data in code	GoldenPricingTest#multiCountry, Liquibase seeds
P-08	Every business rule has a test that has been seen failing	The task cycle (§10), PIT score of at least 80%
P-12	No claim without evidence	HANDOFF.md §3
P-15	A smaller scope never lowers the standard: no TODO, placeholder or dead code	check-docs, review

The other articles fix the language split (English for engineering, Spanish by default for users), the bilingual delivery documents, governed documentation, comments that explain why, and repository READMEs that answer five questions.

3.2 SPEC, business rules and acceptance criteria

orderflow-engine-docs/spec/SPEC.md opens with four guarantees, each with its mechanism:

#	Guarantee	Mechanism
G-1	The quoted total is the confirmed total	A sealed, insert-only quote. The order references it and stores no amount
G-2	No floating point for money	A BigDecimal value object, scale and rounding from country data, rounding per line
G-3	A 201 means the order exists; retries never duplicate it or consume credit twice	Order, credit, idempotency key and outbox in one ACID transaction, the reply after commit
G-4	Push and ERP effects are never lost and never applied twice	Transactional outbox, leased relay with retries, consumers deduplicating by event_id

Then come the scope (§1, with an explicit in and out table), the glossary, and the business rules:

Rule	Subject
BR-00	Cart normalization: merge duplicate SKUs, sort by SKU, reject invalid quantities, empty carts and unknown SKUs
BR-01	The order in which the levers apply, with the reason for each step
BR-02	Tier discounts: marginal brackets, rounded once per line, SKU rule over family rule
BR-03	Combos: applications, discount, one entry per combo, largest-remainder allocation
BR-04	Gifts: condition, separate gift line, reference value, country gift tax policy
BR-05	Tax per line and the total identities every breakdown must satisfy
BR-06	Credit: advisory at quote time, conditional consumption at confirmation
BR-07	Quote lifecycle: OPEN, CONFIRMED, EXPIRED, derived and never stored

BR-02 and BR-03 carry worked examples computed by hand before any code. They became the fixtures of golden tests 1 and 4, so the specification and the tests share the same numbers.

The acceptance criteria are 52 rows, AC-01 to AC-53. AC-22 was removed from scope, and IDs are never renumbered. Each row names the rule it serves, the task that delivers it and the test class that proves it.

3.3 Traceability

SPEC §14 links the assignment to the code in three tables:

Table	Link it keeps
§14.1 Assignment coverage	Each assignment requirement (REQ-A1-1 … REQ-A1-6, REQ-A1-T1 … T6, REQ-A2-1 … 4, REQ-A3, REQ-D, REQ-F, REQ-P) to its acceptance criteria
§14.2 Acceptance	Each criterion to its rule, its task and the test that verifies it

criteria	
§14.3 Beyond the assignment	Every item the assignment did not ask for, with its tag (quality, stability, scalability), its benefit and its origin

The chain is always the same: assignment bullet → REQ → AC → BR or SPEC section → task → test class. A measured review on 2026-10-15 found 21 of 21 assignment bullets specified (cross_reference_check.py, recorded in the HANDOFF).

3.4 Documentation governance and its automatic check

Documentation follows written rules (orderflow-engine-docs/governance/documentation-rules.md): one question per document, a document map, front matter on every document, delivery documents in English and Spanish with the same headings, and internal technical documents in English. A rule that no check enforces decays (HANDOFF trap TR-03), so the rules are enforced by tools in the orderflow-engine-tools repository:

Check	What it fails on
docs-check/check-docs.mjs	A Markdown file outside the allowed locations; a file missing from the document map; a language pair missing or with a different heading count; missing or invalid front matter; a relative link that is broken or leaves its repository; an ADR above 500 words; a missing required heading (for example the four topics of the API README); i18n bundles with different keys; a progress chart that does not match the HANDOFF task table; a commit line in TASKS that breaks the conventions; a generated Word or PDF whose source changed after it was built
integrity/check-integrity.mjs	Missing ignore files or .gitattributes, line-ending rules, Docker files, .env.example, and secrets in any of the six repositories
consistency/	Counts, cross-references, language and pair parity across documents

4. The plan

orderflow-engine-docs/spec/TASKS.md turns the specification into tasks. It also fixes nine global constraints (GC-1 … GC-9), the task cycle, a review focus with nine risky edges (RF-1 … RF-9), the day plan and the cut line. orderflow-engine-docs/spec/plan.md lists every technical decision with its rejected alternatives (DEC-01 … DEC-20) and the risk register (RISK-01 … RISK-07).

4.1 Tasks and phases

Task	Title	Phase
T-00	Specification, governance, README, ADR draft	1. Preparation
T-01	Scaffold API and workspace test command	2. Scaffold
T-02	Money and currency policy	3. Core
T-03	Pricing pipeline and golden harness	3. Core
T-04	Tier discounts	3. Core

T-05	Combos and allocation	3. Core
T-06	Gifts and gift tax policy	3. Core
T-07	Rounding and multi-country	3. Core
T-08	Schema and seed data	3. Core
T-09	Error handling and API i18n	3. Core
T-10	Quote use case and endpoints	3. Core
T-11	Order confirmation	3. Core
T-12	Invariants under concurrency and failure	3. Core
T-13	Order history	3. Core
T-14	Outbox relay	3. Core
T-15	Retention housekeeping	3. Core
T-18	Wire the levers into quotes	3. Core
T-16	Final documentation and verification	3. Core
T-17	Web client: summary and full flow (optional)	4. Web
T-19	Fixes after the T-17 preview	4. Web
T-20	Seed Guatemala and Peru	4. Web

Phase	Tasks	Closed on	Merge commit on `main`
1. Preparation	T-00	2026-10-14	chore: close preparation phase (T-00)
2. Scaffold	T-01	2026-10-15	chore: close scaffold phase (T-01)
3. Core	T-02 … T-16, T-18	2026-10-15	chore: close core phase (T-16)
4. Web	T-17, T-19, T-20	2026-10-17	chore: close web phase (T-17)

4.2 Day plan and cut line

Day	Planned tasks
D1	T-00 … T-03
D2	T-04 … T-07
D3	T-08 … T-10
D4	T-11 … T-14
D5	T-15, T-18, T-16, then T-17 slice by slice, then T-19 and T-20

The MVP cut line sits after T-16. Everything the assignment requires is above it. T-17, the optional front, is below it and was split into thematic slices that are each safe to cut: after every web slice the client builds, runs in Docker and passes its tests, so stopping after any slice leaves no route, key or endpoint half wired. The five-day window is risk RISK-07 of the plan, and progress was to be checked against the plan at the end of day 4 (open decision OD-02).

4.3 How the plan met reality

The merge history of the API repository shows the order actually taken. On 2026-10-15, T-02 and T-03 (money and pipeline) merged first, then T-08, T-09, T-14, T-15 and T-10 (schema, errors, outbox, housekeeping, quotes), then T-11, T-12 and T-13 (confirmation, its invariants, history). The levers T-04 … T-07 followed, then T-18 and T-16. The

persistence and confirmation path was therefore built before the levers. TASKS records the consequence: T-10 had been built before T-04 … T-06, so its quote path loaded prices and tax rates only. T-18 was added to wire the seeded levers into quotes, with QuoteLeversIT replaying the tier, combo and gift cases through POST /api/v1/quotes, before T-16 closed the core.

Two tasks were added after T-17. T-19 collects the fixes found by walking the full stack: product and customer names in the tenants' language, a quantity cap of 1,000,000, currency codes next to amounts, a remembered language choice, restart by Docker and an end-to-end step in test-all. T-20 adds Guatemala and Peru as data only.

5. The pricing engine (A1) step by step

5.1 The money type

Aspect	Decision
Type	Money(BigDecimal amount, CurrencyPolicy policy), an immutable value object. float and double are banned from pricing.domain by an ArchUnit rule
Scale	Data: the currency's row (COP 0; USD, GTQ and PEN 2). Never java.util.Currency, which reports COP with 2 decimals (HANDOFF trap TR-01); another ArchUnit rule forbids it in the domain
Rounding mode	Data: the country's row. HALF_UP in the four seeded countries
Storage	Money is NUMERIC(19,4), rates are NUMERIC(7,6). A price with more decimals than its currency allows is rejected when it is loaded (PriceScaleValidationIT)
Comparison	Always compareTo, never equals, because equals also compares the scale
API	Money is a JSON string at the currency's scale, and the ISO currency code appears once per body. The web client formats that string and never turns it into a number

5.2 Normalization and the pipeline

The engine is one pure function: Breakdown PricingEngine.price(Cart, PricingSnapshot). The application layer resolves the snapshot first: unit prices, active levers, tax rates and country policy at one point in time. The engine sees no database, clock or framework, so it is deterministic and can be pinned by golden tests (P-06).

Before pricing, the cart is normalized (BR-00): duplicate SKUs are merged, lines are sorted by SKU, and invalid input fails fast with a stable code: a quantity of zero or less, non-integer or above 1,000,000 → 400 INVALID_QUANTITY; an empty cart → 400 CART_EMPTY; an unknown SKU or one of another tenant → 422 UNKNOWN_SKU. A gift quantity derived above the same bound is also a 400, computed with overflow-checked arithmetic so it never wraps or answers 500.

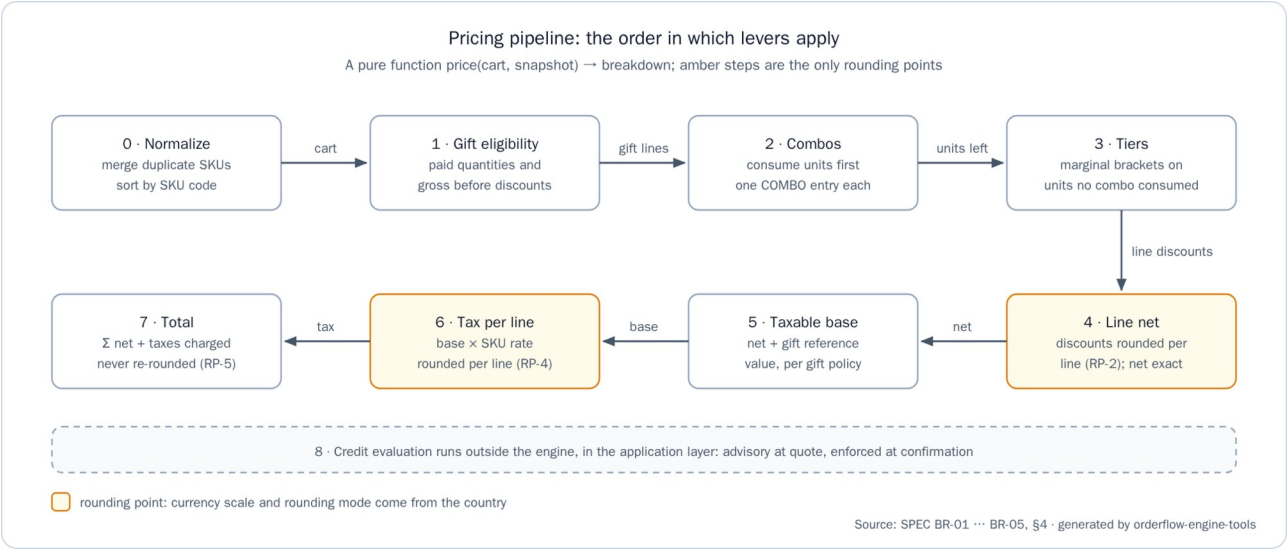


Figure 1. Pricing pipeline: the order in which levers apply

5.3 Lever order and its reasons

Step	What	Why here
1	Gift eligibility, on paid quantities and the gross subtotal before discounts	A gift is triggered by what the customer buys and never changes what they pay for product
2	Combos	A combo is a negotiated bundle price. It consumes its units first, so they cannot also get a tier discount
3	Tier discounts on the units no combo consumed	The generic volume incentive applies only to what the bundle did not price
4	Line net = gross − discount components	Commercial discounts are settled before tax
5	Taxable base	The assignment requires tax on the base after commercial discounts
6	Tax per line	Each SKU has its own rate
7	Total	It needs every previous step
8	Credit evaluation, in the application layer	It needs the final total and is not a price lever, so it stays outside the pure engine

Each lever has a precise rule:

- **Tiers (BR-02).** A rule targets a SKU or a family and defines brackets {from, percent}. The discount of a line is computed exactly and rounded once per line, never once per bracket segment (review edge RF-1).
- **Combos (BR-03).** Applications = the floor of the smallest qty / minQty across the combo's SKUs. Discount = list value of the consumed units − combo price × applications. A combo that would not lower the price consumes nothing. The discount is split across the participating lines in proportion to their consumed gross value by the largest-remainder method, with ties broken by SKU order, so the parts always add up exactly.

- **Gifts (BR-04).** A gift is a separate line with its quantity, GIFT:<id> origin and price 0. Its reference value is quantity × list price. The country's gift tax policy decides whether that value is taxed (TAXED) or not (EXEMPT).

Discarding the alternative orders was explicit in the ADR: applying tiers before combos would discount the same units twice.

5.4 Rounding points

Rounding happens per line only:

Point	Where	Rule
RP-1	Unit price × quantity	Exact, because prices are already at currency scale
RP-2	Each discount component of a line (tier, combo allocation)	Rounded to currency scale. A component that rounds to zero is not emitted
RP-3	Net = gross − components	Exact
RP-4	Tax of a line	Rounded to currency scale
RP-5	Order totals	Sums of rounded line values, never rounded again

Why per line: an electronic invoice carries tax per line, so per-line rounding keeps the order consistent with the invoice it produces. Rounding once per tax-rate group was rejected because line values would disagree with the totals, and rounding only the final total was rejected because it hides cent drift and cannot be audited line by line.

Every breakdown satisfies the identities of BR-05, and every golden test asserts them: gross subtotal = Σ paid gross; Σ order discounts = Σ line tier discounts + Σ line combo allocations; line net = gross − discounts − allocations; taxable base = Σ paid net + Σ gift taxable base; taxes = Σ line tax; **total = Σ net + taxes**.

5.5 The six golden tests

A pricing case is a folder under src/test/resources/golden/ with request.json, snapshot.json and expected.json, tagged @Tag("golden"). GoldenPricingTest checks each folder twice: against its expected breakdown and against the identities above. All amounts below are Colombian pesos (COP, scale 0) from the fixtures.

Test	Assignment case	What it proves
GT-01 gt01-two-tier-brackets	A discount in two brackets	25 colas of 350 ml at 2,500, brackets 10–19 → 5% and 20+ → 10%: 1,250 + 1,500 = 2,750 in one TIER entry. Tax 19% of 59,750 = 11,352.50 → 11,353. Total 71,103. Marginal brackets, rounded once per line
GT-02 gt02-combo-no-loose-discount	A combo whose products are not a loose discount	21 colas and 10 chips. The combo of 2 colas and 1 chips at 5,000 applies 10 times and consumes 20

		colas and 10 chips: one COMBO entry of 18,000. The cola left over stays in the first bracket, so no TIER entry appears, although a tier rule is seeded on the same SKU and 21 colas alone would reach the 10% bracket. Total 60,622
GT-03 gt03-gift	A gift with its quantity	25 chips and 3 waters. The rule "one water free for every 10 snacks, repeatable" adds a separate gift line of 2 waters with GIFT:<id> origin. Paid lines are unchanged. Colombia taxes gifts. Total 53,175
GT-04 gt04-combo-with-taxes	A combo with taxes	4 colas and 2 chips. The combo applies twice: a discount of 3,600, split 2,647 and 953 by largest remainder. Taxes 1,397 at 19% and 132 at 5%, so 1,529 enters the total: 11,529
GT-05 gt05-credit-exceeded	A credit order over the limit	A credit quote of 40 waters and 60 colas of 1.5 l totals 442,680 against 400,000 available: fits=false, shortfall 42,680. It runs through the quote use case (GoldenQuoteTest#gt05), because credit is outside the pure engine. Its companion CreditLimitIT confirms it: 409 CREDIT_LIMIT_EXCEEDED, with no order and no ledger row
GT-06 QuoteConfirmRegressionIT	Quote and confirm give the same total	Against PostgreSQL and the seeds. Between quote and confirmation the chips price changes and the levers are deactivated. The confirmed total equals the quoted total, **and** a fresh quote of the same cart differs, which proves the test can fail. A second case shows that the sealed quote keeps its combo, tier and gift

More cases run in the same harness: both gift tax policies, .5 rounding edges, the same cart in Colombia (scale 0) and Ecuador (scale 2), and carts without levers. TierPrecedenceTest proves that a SKU rule beats a family rule.

6. The idempotent order (A2) step by step

6.1 The sealed quote

POST /api/v1/quotes normalizes the cart, resolves the snapshot, runs the engine and inserts the quote, its lines and their discount components. Every line keeps the unit price and tax rate in force at that moment, and the rounded taxable base and tax. These rows are insert-only: no UPDATE statement exists for quotes.

POST /api/v1/orders writes no amount. The order row holds its number, its quote, its customer and its confirmation time. Its breakdown is rendered from the quote lines, so every figure exists once, and the confirmed total is the quoted total by construction. The engine is never run again at confirmation: if it were, a deployment that changed the engine would change every open quote. The quote's state is derived, never stored: CONFIRMED when an order exists, otherwise EXPIRED after expiresAt, otherwise OPEN.

Rejected alternative	Why
Copy the breakdown into the order	Two copies can diverge
Recompute at confirmation and compare	It fails whenever a price or lever changes
Lock prices while quotes are open	It blocks catalog updates

The trade-off is explicit: a price change applies to new quotes only, and an open quote keeps its price until it expires, 15 minutes by default. That is the risk recorded in the ADR.

6.2 One transaction

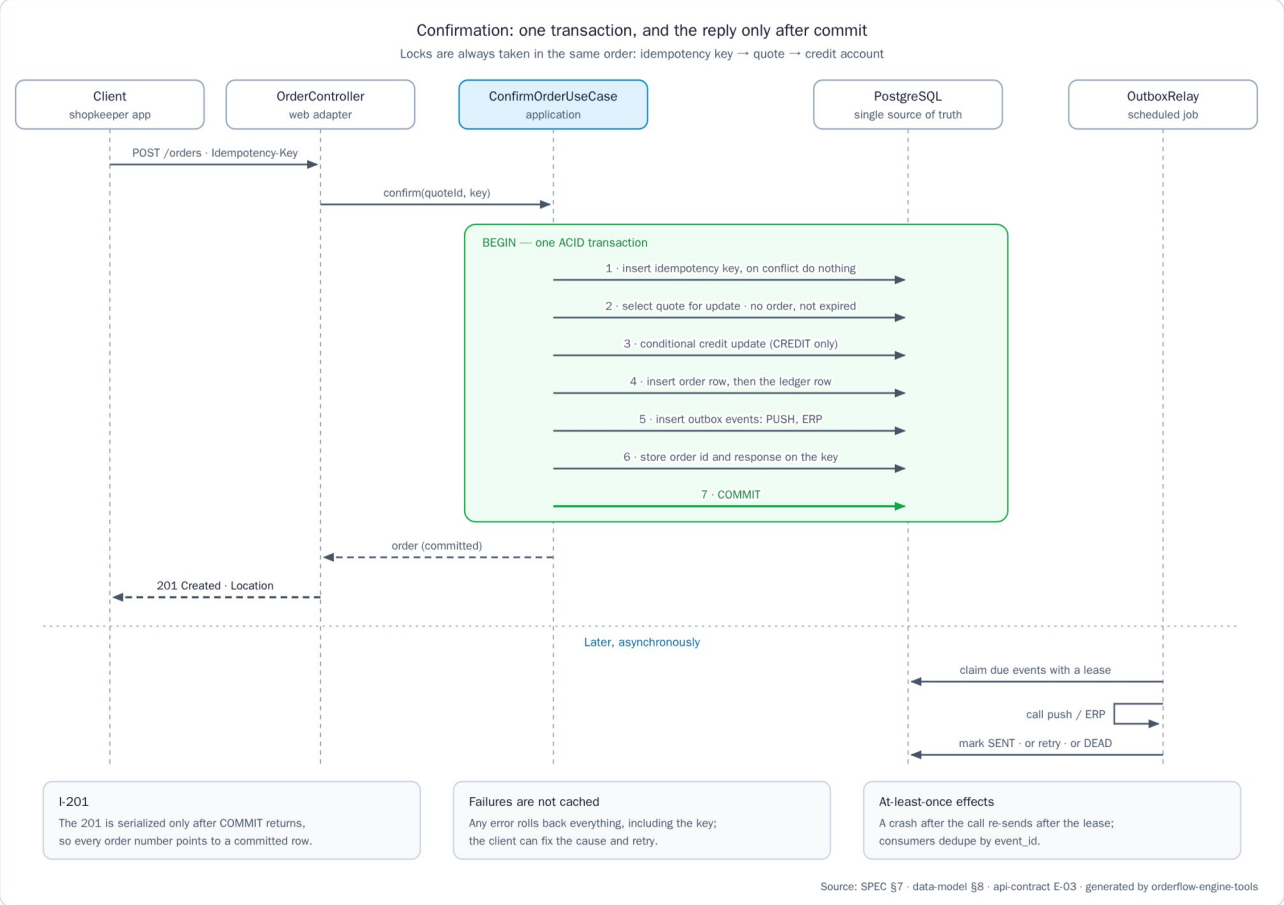


Figure 2. Confirmation: one transaction, and the reply only after commit

A confirmation is one ACID transaction in PostgreSQL, under READ COMMITTED with explicit row locks and a lock_timeout. Locks are always taken in the same order, idempotency key → quote → credit account, so concurrent confirmations cannot deadlock.

Step	Action	Guarantee
1	Insert the idempotency key with ON CONFLICT DO NOTHING, then read it back	One owner per key. A concurrent duplicate waits on the uncommitted unique entry
2	Lock the quote with SELECT … FOR UPDATE; check its owner, that it has no order and that it has not expired	One confirmation per quote
3	On credit, consume it with a conditional update	Credit is never consumed over the limit
4	Insert the order (unique per quote), then its ledger row on credit	No order without its quote, no second order for one quote
5	Insert one outbox event for push and one for the ERP	Effects are recorded atomically with the order
6	Store the 201 body on the idempotency row	Replays return the same answer
7	Commit, and only then serialize the 201	Invariant I-201

The technology choices follow from this transaction (plan DEC-01 … DEC-12): Spring Boot on the servlet stack, because a reactive stack complicates the single blocking transaction; a modular monolith, because splitting order, credit, key and outbox into services would need a saga; PostgreSQL, because an in-memory store cannot prove atomicity or row locks; JdbcClient with explicit SQL instead of JPA, which hides locks and flush order; and Testcontainers instead of H2, whose locking and SQL dialect differ. @Transactional appears only on application use cases, which an ArchUnit rule enforces.

6.3 The idempotency key

Idempotency-Key is required on POST /api/v1/orders and must match ^[A-Za-z0-9_-]{8,64}$. The request hash is SHA-256 of method, path, customer id and canonical JSON body. The semantics follow the IETF draft "The Idempotency-Key HTTP Header Field":

Situation	Response
Same key, same body, already completed	The stored response, byte-identical, with Idempotent-Replayed: true
Same key, different body	422 IDEMPOTENCY_KEY_REUSED
Same key while the first request is still running	Waits up to lock_timeout, then replays; 409 IDEMPOTENCY_KEY_IN_PROGRESS if the timeout passes first
Missing or malformed key	400 IDEMPOTENCY_KEY_MISSING or IDEMPOTENCY_KEY_INVALID
A failed request (validation, expired quote, credit exceeded)	Not cached: the rollback releases the key, and the client can retry after fixing the cause
A new key on a confirmed quote	409 QUOTE_ALREADY_CONFIRMED, with the existing order number

Keys are kept for 24 hours. After that, a retry still receives QUOTE_ALREADY_CONFIRMED with the order number. The web client completes the guarantee on its side: one key per quote, created at the first tap of "Confirm order", stored in the browser and reused on every retry, with the button disabled while the request is in flight.

6.4 The credit ledger

At quote time a credit quote reports credit {fits, available, shortfall}, where available = limit − consumed and a total equal to the available amount fits. Nothing is reserved. At confirmation, inside the transaction:

1. UPDATE credit_accounts SET consumed = consumed + :total … WHERE … AND consumed + :total <= credit_limit.
2. If no row is updated, the answer is 409 CREDIT_LIMIT_EXCEEDED and the rollback leaves no order and no ledger row.
3. Otherwise a ledger row (order_id, amount) is inserted right after the order. The ledger is append-only, with one row per order at most.

The database backs the code: a CHECK keeps consumed between 0 and the limit even if application code were wrong, and consumed = Σ ledger amounts always holds. CreditLimitIT and ConfirmOrderConcurrencyIT prove it.

6.5 The outbox

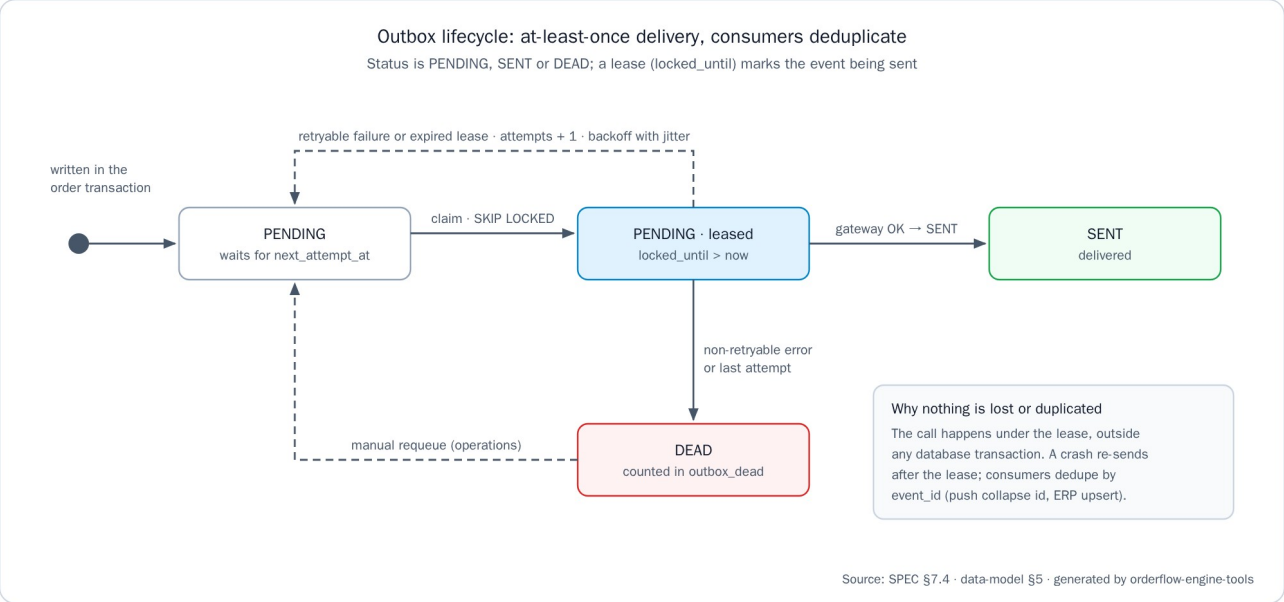


Figure 3. Outbox lifecycle: at-least-once delivery, consumers deduplicate

Stage	Behavior
Write	One event per destination (PUSH, ERP), written in the order transaction. Its id is the event_id
Claim	The relay polls every second by default and leases due PENDING events with FOR UPDATE SKIP LOCKED for 30 seconds, so several instances can run at once
Send	Outside any database transaction, passing the event_id downstream
Mark	Success → SENT. A retryable failure is retried with exponential backoff

	and full jitter, from 2 seconds up to 5 minutes. After 8 attempts, or at once on an error that retrying cannot fix such as an ERP 4xx → DEAD, counted by the outbox_dead gauge and requeued by the procedure in the API README
Never lost	If the service stops between send and mark, the lease expires and the event is sent again
Never applied twice	Consumers deduplicate by event_id: push uses it as the collapse id, and the ERP upserts by order number. The fake gateways of the repository deduplicate the same way and can be told to fail
External system down	The order stays confirmed. The effect waits in the outbox, outbox_pending shows the backlog, and delivery resumes on recovery

Publishing after commit was rejected because a crash loses the event; two-phase commit, because push and the ERP cannot join it; change data capture, because it is too much infrastructure for this scope. The tests are OutboxWriteIT (exactly one push and one ERP event per order) and OutboxRelayIT (outage recovery, lease expiry with deduplication, DEAD).

6.6 Consistency model and what it sacrifices

Scope	Model	Sacrificed
Order, credit, idempotency key, outbox events	Strong: one ACID transaction in one database	Availability while the database is down. Confirmations of the same customer serialize on its credit row. The write path depends on a single database
Push and ERP	Eventual: at-least-once delivery, deduplicated by consumers	They lag behind the confirmation by the poll interval plus any retries

6.7 The "three releases": root-cause hypotheses and the test that closes each

The assignment reports orders that were confirmed, given a number, and then missing. The cause was never stated, so each plausible root cause was listed (SPEC §7.6), answered by a design rule, and closed by a test that fails if the rule is broken:

Plausible root cause	Design answer	Test that closes it
The response was sent before the commit	The 201 is serialized only after the commit (I-201)	ConfirmOrderCommitOrderingIT: when the answer is written, no transaction is open and another connection already reads the order. A failure raised by the commit itself, through a deferred constraint trigger, is never answered with a 201, and a retry then confirms once
@Transactional was bypassed by self-invocation	The transaction boundary is a separate use-case bean, enforced by	TransactionBoundaryArchTest, ConfirmOrderFailureInjectionIT

	ArchUnit	
The history was read from a replica or from the ERP	Order detail and history read the primary tables synchronously	OrderReadYourWritesIT: every 201 order is immediately readable by id and in the history
The order was written asynchronously after replying	The order row is written in the request transaction; only the effects are asynchronous	ConfirmOrderFailureInjectionIT: a database trigger makes one of five writes fail before the commit (credit update, order, ledger, outbox, key). Nothing persists and the key is released; the retry with the same key creates exactly one order. When the answer is lost after the commit, the retry returns the same order number, byte for byte, as a replay
A lost update or a duplicate on a double tap	Unique idempotency key, unique quote per order, conditional credit update	ConfirmOrderConcurrencyIT: 6 concurrent requests with one key, sent while the credit row is held, all get a 201 with the same body, only one is not a replay, and there is one order, one ledger row and two outbox events. Two confirmations that together exceed the limit: exactly one succeeds

The web client closes the same defect on its side: it keeps no copy of orders, and My orders reads the history on every visit, so a committed order cannot be hidden by a client cache (AC-52, proven by history-read-your-writes.spec.ts and the end-to-end flow).

7. The ADR (A3)

The assignment asks for one ADR of at most one page, so the delivery has exactly one: orderflow-engine-api/docs/adr/0001-order-pricing-and-confirmation.md, "Sealed quotes and a single-transaction confirmation", with its Spanish pair. It uses the header table of MADR 4.0.0 (status, date, decision makers, confidence) and the four sections A3 requires.

Section	**Content in short**
Decisions and discarded alternatives	Sealed quote referenced by the order (not copying, not recomputing); BigDecimal rounded per line, combos before tiers (not long minor units, not rounding the total, not tiers first); one PostgreSQL transaction with locks key → quote → credit and the 201 after commit (not in-memory, not asynchronous write or saga, not keys in a cache); transactional outbox with lease and backoff (not publishing after commit, not two-phase commit); hexagonal monolith, explicit SQL, separate repositories (not microservices, not JPA, not a monorepo)
Not implemented, on purpose	Authentication, real push and ERP clients, and what was not asked for
Code review	No float or double; compareTo for money; no UPDATE on quotes;

checklist	@Transactional only on use cases; lock order; 201 after commit; no cached failure; relay outside transactions; every lever with a golden test seen failing; user text only in i18n
Risk and mitigation	A sealed quote honors a wrong price until it expires. Mitigation: a 15-minute TTL per country, then revocation (OPEN → REVOKED, 409). Revisit if an incident affects more than 100 quotes or the TTL exceeds one hour

The one-page limit is enforced twice: check-docs fails above 500 words per language, and the Word and PDF build (Arial 12, A4, 2 cm margins) fails above one page. The ADR entered as a draft in T-00 (docs(api): add ADR-0001 draft (T-00)) and was accepted in T-16 (docs: accept ADR and record evidence (T-16)). Every other decision, with its rejected alternatives, lives in orderflow-engine-docs/spec/plan.md, so the ADR stays within one page.

8. The web client and the full stack

8.1 From the one-hour plus to the full flow

The assignment's plus is one order-summary screen in at most one hour. That screen was built first, alone, so its time could be measured: 68 minutes, which already exceeds the hour once its tests and its container are counted. On 2026-10-15 the scope of the front was extended on purpose, as a product-owner decision recorded in SPEC §1 ("Deviation"), to the domain flow the assignment describes:

Route	Screen	API operations
/cart, also /	Cart: product grid, quantities, cash or credit	E-07 (product list), E-01 (quote)
/quotes/:quoteId	Order summary, the assignment's screen	E-02, E-03 (confirm)
/orders/:orderId	The confirmed order	E-04
/orders	My orders	E-05

The extension was contained: it started only after T-16 was merged, added no business rule, never computes money (every amount is an API string), and added a single read-only API operation, GET /api/v1/products (E-07). A store picker sends a seeded tenant and customer as headers instead of a login (A-06). Spanish is the default language and English is complete; the ES/EN switch changes the language at run time and the browser remembers the choice.

8.2 Prevention at component level

Each defect the assignment names is owned by one component, so a new screen inherits the prevention instead of solving it again:

Defect	Component	Rule
Truncated text	ui-text	The only way a screen prints text. It always wraps, never uses ellipsis or a line clamp, and an amount never splits
Controls hidden by the system bar or the	ui-action-bar	Sticky to the bottom, padded with the safe-area insets, lifted above the

keyboard		keyboard by following visualViewport; every control at least 44 × 44 px
Grid images scaled differently	ui-product-image	One aspect ratio with object-fit: contain, the only way to draw a product image, with a placeholder when one is missing

The specs measure the real layout in headless Chromium at 768 × 1024 and 375 × 812 with very long product and store names, because jsdom has no layout engine and a wrapping check passes there for any layout (HANDOFF trap TR-11). A script also fails the build if any template holds literal text.

8.3 The full stack and the end-to-end flow

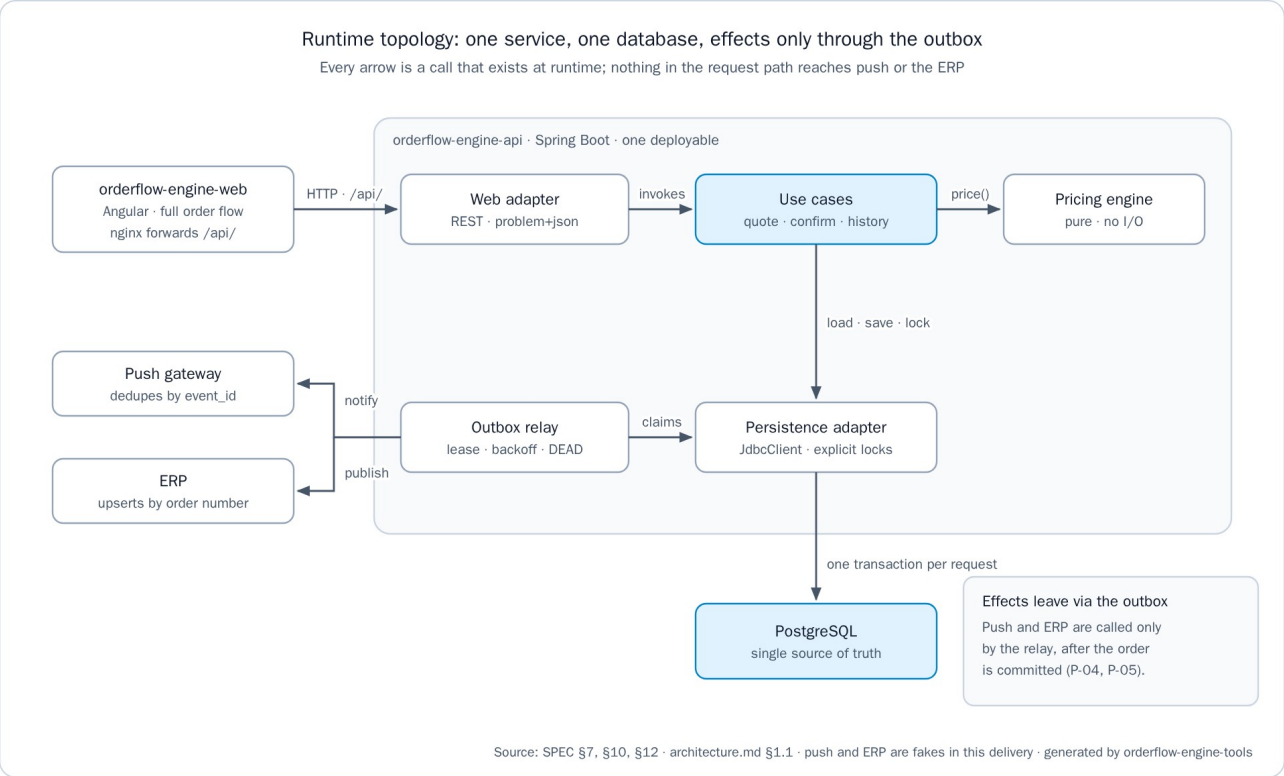


Figure 4. Runtime topology: one service, one database, effects only through the outbox

The web client's nginx forwards /api/ to the API, so the browser sees one origin and the API needs no CORS configuration. The orderflow-engine-infra repository starts PostgreSQL, the API and the web client with one docker compose up --build, each service with restart: unless-stopped. Its test-all runs, in order, the API verify, the web specs in their container, an end-to-end Playwright flow against the full stack (cart → quote → confirm → order → My orders, in Spanish and in English, under its own compose project and ports), the documentation check and the integrity check.

9. Countries and seeds

Countries, currencies, tenants, catalogs, levers, customers and credit accounts are data, written as Liquibase changesets in orderflow-engine-api/src/main/resources/db/changelog/. Each changeset has a rollback, each changelog sets a logicalFilePath, and LiquibaseRollbackIT proves that every

changeset rolls back to an empty schema and applies again. The seeds are demonstration data chosen to cover both scales and both gift tax policies, not a statement of real tax law.

Tenant	Country policy	Catalog and levers	Customers and credit
co-alpha	COP scale 0, HALF_UP, gifts TAXED, TTL 15 min	Colas, water and chips; 19% and 5% tax. Tier on the 350 ml cola (10 → 5%, 20 → 10%); combo of 2 colas and 1 chips at 5,000; one water free for every 10 snacks	One customer with a 500,000 limit, one paying cash
ec-alpha	USD scale 2, HALF_UP, gifts EXEMPT, TTL 15 min	15% tax; family tier on colas (12 → 4%, 24 → 8%); a combo at 3.99; a gift from a 50.00 gross subtotal	A credit account with a 1,500.00 limit
gt-alpha	GTQ scale 2, HALF_UP, gifts EXEMPT, TTL 15 min	12% tax; tier on the 350 ml cola (12 → 5%, 24 → 10%); one water free for every 12 colas	A 5,000.00 limit, and a store paying cash
pe-alpha	PEN scale 2, HALF_UP, gifts TAXED, TTL 15 min	18% tax; combo of 2 colas and 1 chips at 6.00; one water free from a 100.00 gross subtotal	A 2,000.00 limit, and a store paying cash

The seeds grew in three changelogs. 2026-10-15-006-seed-co-and-ec.yaml seeded Colombia and Ecuador in T-08. 2026-10-16-001-seed-names-in-spanish.yaml renamed products and customers to the tenants' language in T-19, because an applied changeset is never edited. 2026-10-16-002-seed-gt-and-pe.yaml added Guatemala and Peru in T-20 with no Java change, which is itself the proof that no currency, scale, rate or rule is hard-coded (P-07). ProductContractIT lists GTQ and PEN at scale 2, and QuoteLeversIT quotes a Guatemalan cart (tier and untaxed gift) and a Peruvian cart (combo and taxed gift) whose totals were computed by hand. Order numbers come from one sequence per country, such as CO-000042. The web store picker lists the eight seeded stores, two per tenant.

Catalog administration is out of scope by design (SPEC §1): a new store, product or lever is a new changeset.

10. How quality was proven

Each task is built with the same cycle (TASKS, "Task cycle"): write the failing test and see it fail for the expected reason; implement the minimum to pass; **prove the test bites** by breaking the guard once (flip a rule, remove a lock, skip a rounding) and watching the test fail; run the full verification; then open the pull request with the verification output. Recorded examples of a guard broken on purpose: each domain purity rule failed with a probe class, the JaCoCo gate failed with an uncovered probe class (HANDOFF at the scaffold close), and a failing web spec made test-all fail in PowerShell too (the pull request that added the web client to the full stack).

Practice	How it was applied	Evidence (HANDOFF §3)
Unit and golden tests	JUnit 5 and AssertJ; JSON fixtures for full breakdowns with a fixed clock and fixed ids	333 unit tests green, the six golden tests among them
Integration on a real database	Testcontainers PostgreSQL 17, never H2	308 integration tests green
Concurrency and failure injection	Concurrent requests on one key, a held credit row, database triggers that fail a write, a deferred trigger that fails the commit, a filter that loses the answer after commit	ConfirmOrderConcurrencyIT, ConfirmOrderFailureInjectionIT, ConfirmOrderCommitOrderingIT
Coverage	JaCoCo gate of at least 90% of lines and branches on pricing, which fails ./mvnw verify	100% of lines and branches in pricing.domain and pricing.domain.lever; 99.2% of lines and 98.0% of branches in the whole service
Mutation testing	PIT in the optional profile ./mvnw -Pmutation verify, threshold 80%	185 of 185 mutations detected (100%)
Architecture rules	ArchUnit: pure domain, no float or double, no java.util.Currency in the domain, @Transactional only on use cases	DomainPurityArchTest, TransactionBoundaryArchTest
Contract and i18n	MockMvc on status codes, problem+json shape and localized texts; every error code has an es and an en message	ProblemDetailContractIT, LocalizedErrorIT, ErrorCatalogI18nTest
Web specs	Headless Chromium inside the test stage of the web Dockerfile; every spec answers through a simulated backend and asserts the exact request	209 specs in 15 files
End to end	Playwright against the full stack, in Spanish and in English	The flow passes 2 of 2
Clean-export verification	From the scaffold phase on, each API pull request was verified on a clean export of its own commit (git archive of the branch head, then ./mvnw verify), so every commit on dev builds on its own; the one documentation-only pull request of the core phase was verified with ./mvnw verify on its commit. Each web pull request ran docker build --no-cache-filter test --target test on a clean export,	HANDOFF §9 … §11

	and each infra and tools pull request ran its own checks	
Workspace verification	From the scaffold phase on, test-all passed before each phase close. Before the web phase closed, it passed on fresh clones from GitHub of dev of every code repository and main of the documentation repository, on 2026-10-17	test-all: OK (API verify, web tests, end-to-end, documentation check, integrity check)

The test suite grew with the work: 3 tests at the scaffold close (the two domain purity rules and the health check), 310 unit and 287 integration tests at the core close, and 333 unit and 308 integration tests at the web close, as successive versions of the HANDOFF record.

11. The git history

11.1 Repositories and branches

Repository	Role	Long-lived branches
orderflow-engine-api	**The deliverable**: service, tests, README, ADR	main, dev
orderflow-engine-web	The Angular client	main, dev
orderflow-engine-infra	Full-stack compose and test-all, with the end-to-end suite	main, dev
orderflow-engine-tools	Checks and generators for documents and repositories	main, dev
orderflow-engine-app	Reserved for a future mobile app; not built	main, dev
orderflow-engine-docs	Specification, governance, handoff, diagrams, deliverables	main only

The six repositories were created on 2026-10-14. The golden rule (orderflow-engine-docs/governance/golden-rule.md) states that nothing reaches main except through a pull request. In a code repository, task branches are squash-merged into dev, and main receives dev only when a phase closes, through a merge commit, so every task commit stays reachable from main. The documentation repository keeps only main, because it holds living documentation. GitHub rejects branch rulesets on private repositories of a free plan (HTTP 403), so the protection rules were followed by hand while the repositories are private (HANDOFF trap TR-06).

11.2 Pull requests per phase

By gh pr list --state merged on 2026-10-17, 82 pull requests were merged: 31 in the API, 10 in the web client, 12 in infra, 17 in docs, 10 in tools and 2 in app.

Phase	Task pull requests	Phase-closing pull requests
1. Preparation (T-00)	21	5

2. Scaffold (T-01)	9	2
3. Core (T-02 … T-16, T-18)	20	3
4. Web (T-17, T-19, T-20)	18	4

The web row counts what gh listed on 2026-10-17. The HANDOFF lists a 19th task pull request for that phase: its own record of the phase close, in the documentation repository.

The HANDOFF lists each pull request by number and subject in §8 … §11. The task pull requests of the core phase in the API, in merge order, read as the story of the build:

```
feat(pricing): add money value object (T-02)
feat(pricing): add pricing pipeline and goldens (T-03)
feat(db): add normalized schema and seeds (T-08)
feat(api): add localized problem details (T-09)
feat(outbox): relay order events reliably (T-14)
feat(outbox): purge expired data by retention (T-15)
feat(quoting): add sealed quotes (T-10)
feat(ordering): confirm orders atomically (T-11)
test(ordering): prove confirmation invariants (T-12)
feat(ordering): add order history endpoints (T-13)
feat(pricing): apply marginal tier discounts (T-04)
feat(pricing): apply combos before tiers (T-05)
feat(pricing): add gifts with tax policy (T-06)
test(pricing): cover rounding and countries (T-07)
feat(quoting): price quotes with seeded levers (T-18)
docs: accept ADR and record evidence (T-16)
```

11.3 Conventional commits

Every commit is one line that follows Conventional Commits 1.0.0 and ends with its task: <type>(<scope>): <imperative summary> (T-NN), at most 60 characters, in English (orderflow-engine-docs/governance/commit-conventions.md). Task pull requests are squash-merged, so the pull request title is the commit, and the reason and the verification output stay in the pull request body (task, covered criteria, verification, notes). A phase close keeps its task tag, such as chore: close core phase (T-16). check-docs fails when a planned commit line in TASKS breaks these rules.

12. What was not done and why

Item	Why
Authentication	Out of scope. Tenant and customer come from headers, and a gateway would inject them
Payment capture, cancellation and credit release	Not required
Server-side cart persistence	Not required. The cart travels in the quote request, and the web client keeps it in the browser
Quote revocation	Not required. It is the next step of the ADR's risk mitigation
Catalog and lever administration	Not required. The migrations seed the data. The only

	catalog operation is the read-only product list (E-07)
Tax categories and price history	Not required. Each SKU carries its rate, and each quote line keeps the price and rate in force when it was quoted
Real push and ERP clients	Fakes behind ports prove the delivery guarantees
OpenAPI file and deployment	Not required. The contract is orderflow-engine-docs/spec/api-contract.md
CORS configuration	Not needed: the web client's nginx forwards /api/, so the browser sees one origin
SonarQube or SonarCloud, a CI pipeline, mutation testing in CI	They need a server or CI configuration. The project already produces what each one consumes (the JaCoCo XML report, the commands)
A separate migration job with least-privilege roles, a separate schema repository	Liquibase runs at startup with the owner's credential, and the changelogs live with their only owner; each changelog has a logicalFilePath, so it can move without being applied again
Catalog in several languages	Product names are tenant data in the tenant's language; the ES/EN switch translates the interface
Mobile app	orderflow-engine-app is reserved and empty

Each item that a later team could adopt has a written adoption path, with today's state, the change it needs and its trigger, in orderflow-engine-docs/spec/plan.md §3.

13. Time spent

The measured times come from the "Time spent" table of the API README:

Part	**Measured time**
The take-home (A1, A2, A3), including the specification	Three working days, 2026-10-13 to 2026-10-15
The assignment's optional screen: the order summary	68 min
Its container, the web client in the full stack and the web step of test-all	55 min
The extension beyond the assignment: the product list (E-07), the cart, the idempotent confirmation, the order, the store picker and My orders	3 h 1 min

The web times are working time per slice, building and fixing it, taken from timestamps recorded at each step; review time is not included. The slices were built between 2026-10-15 22:05 and 2026-10-16 06:15 (UTC-5), several of them in parallel. The assignment allows five days; the last phase closed on 2026-10-17.

14. How to verify it yourself in 5 commands

From an empty folder, with Git, the GitHub CLI, JDK 21, Node 18 or later and Docker running (read access to the repositories is needed while they are private):

```
for r in api web infra docs tools app; do gh repo clone ariel5253/orderflow-engine-$r; done
(cd orderflow-engine-api && ./mvnw verify)
(cd orderflow-engine-api && ./mvnw -Pmutation verify)
bash orderflow-engine-infra/scripts/test-all.sh
(cd orderflow-engine-infra && docker compose up --build)
```

#	Command	What it proves
1	Clone the six repositories side by side	The workspace layout that test-all and the full-stack compose expect
2	./mvnw verify in orderflow-engine-api	The single command of the assignment: unit, golden, contract, architecture and integration tests against PostgreSQL, and the coverage gate
3	./mvnw -Pmutation verify in orderflow-engine-api	PIT on the pricing domain, threshold 80%
4	bash orderflow-engine-infra/scripts/test-all.sh	Everything: it ends with test-all: OK (API verify, web tests, end-to-end, documentation check, integrity check)
5	docker compose up --build in orderflow-engine-infra	The running stack: the web client on http://localhost:8081 and the API on http://localhost:8080

With the stack running, choose the store "Tienda Norte", add 25 units of "Gaseosa cola 350 ml", choose "Credit" and press "Get quote": the summary shows golden test 1, a 2,750 tier discount, 11,353 of tax and a total of 71,103 COP. Press "Confirm order", and the order appears first in "My orders" with the same total. On Windows, pwsh -File orderflow-engine-infra/scripts/test-all.ps1 replaces step 4.